CS-301 Microprocessor Based System Design

Course Teacher: Dr.-Ing. Shehzad Hasan

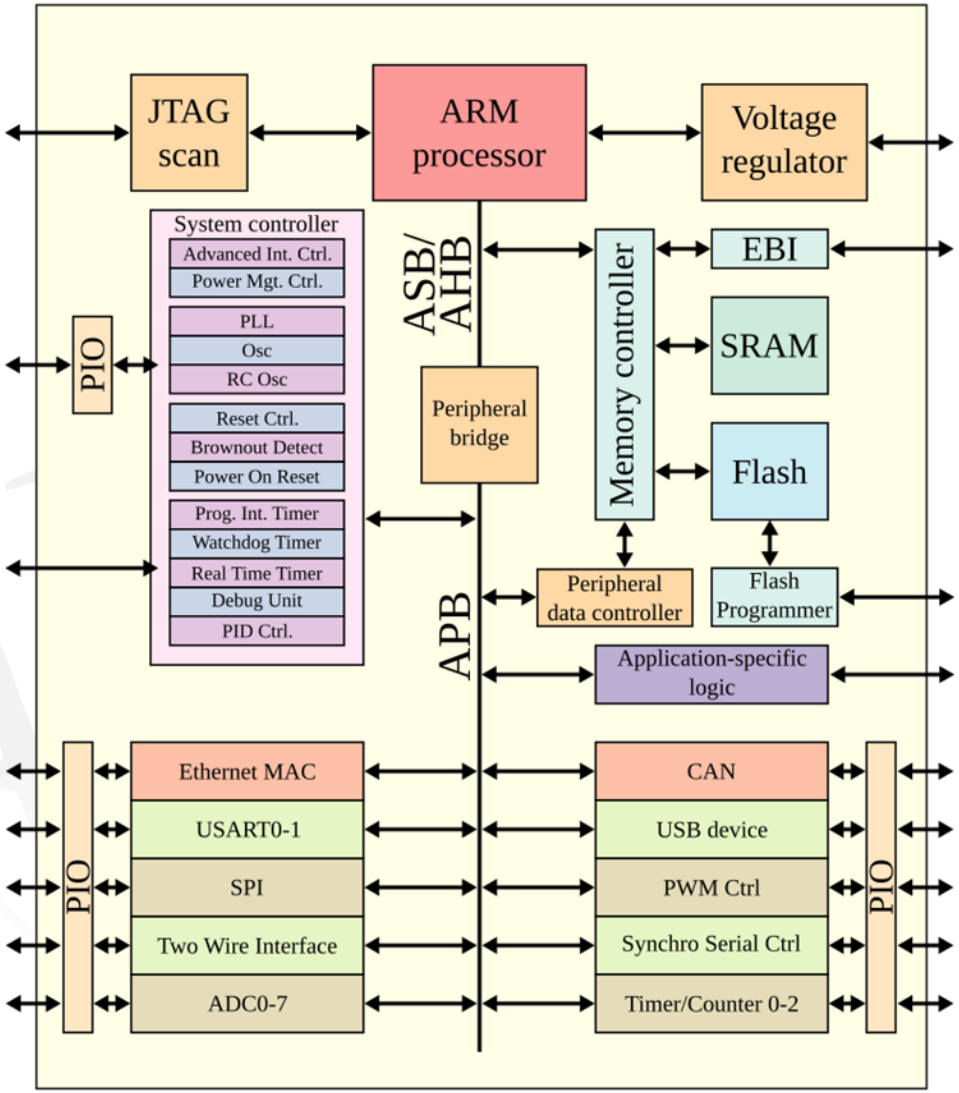
Lecture – 1

Microprocessor

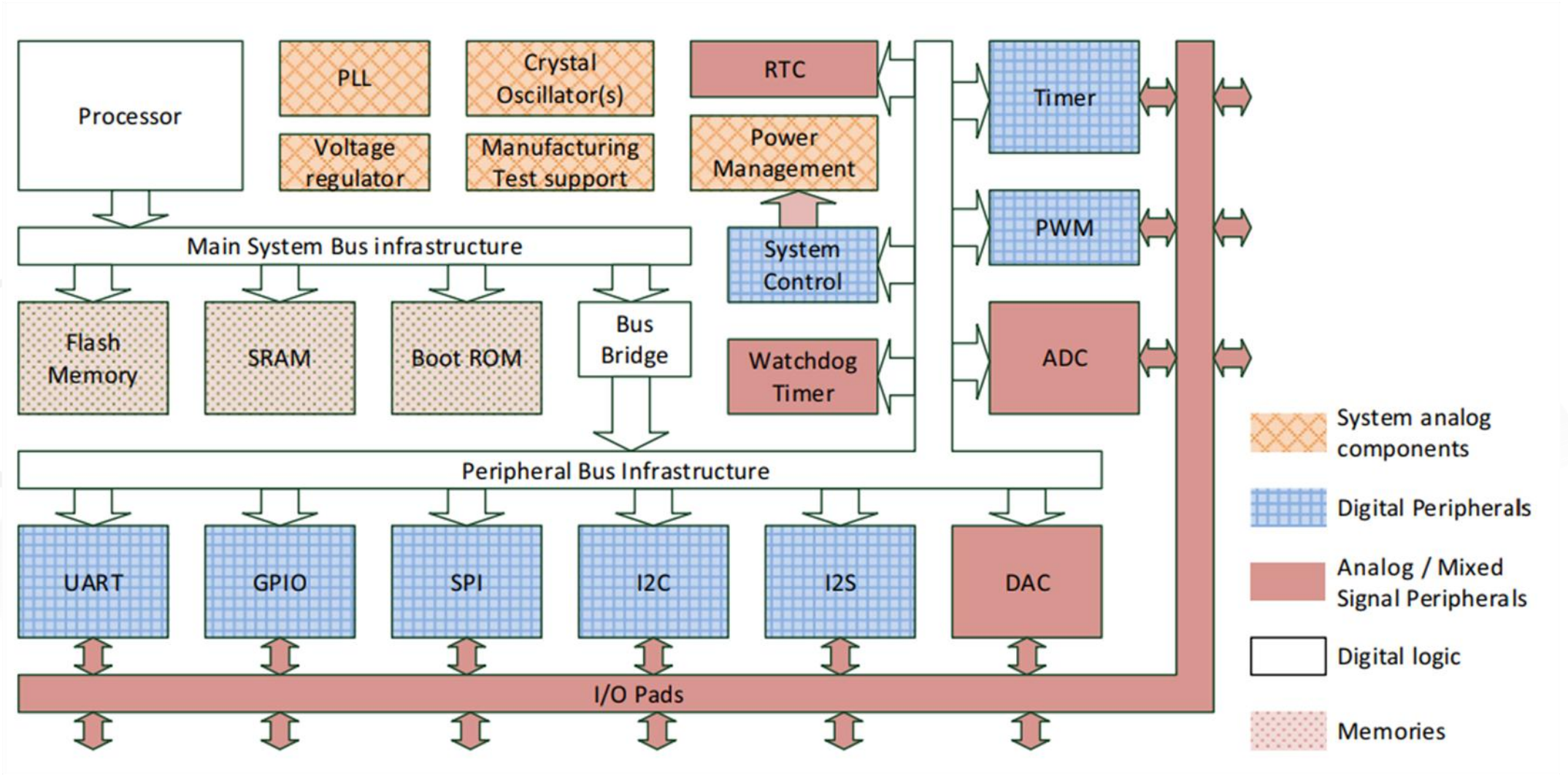
- A microprocessor or central processing unit (CPU) is the "brain" of a computer, responsible for transforming (processing) and transferring data.
- The term "microprocessor" emphasizes that the CPU functions are integrated on a single chip, making it distinct from older or larger systems where the CPU was made up of multiple components.
- A microprocessor-based system integrates a microprocessor with other components like memory, peripherals, I/O devices etc. to create a functional system capable of performing a wide range of tasks across different domains.

System-on-chip (SoC)

- System-on-Chip
  - Integrates microprocessors, memories, peripherals
  - More than 10 billion transistors/cm^2 in 2023
  - Transistors switch between 0 and 1 in picoseconds
  - Consume < 1 femtojoule each time
  - Modern integrated circuits are usually called SoCs
- Applications
  - Low-cost battery-powered mobile devices
  - Medical devices
  - Consumer gadgets
  - Education
  - Warfare



SoC/Microcontroller Components



Classes of Computers

- Personal Mobile Device (PMD)
  - Largest market in terms of revenues
  - Web-based and Media oriented applications
  - Battery powered
  - No fan for cooling
    - e.g. smart phones, tablet computers

Emphasis on
- Energy efficiency
- Size (code size, and hardware) and weight
- Cost (less expensive packaging, optimized memory)
- Responsiveness
  - Real-time constraints (Soft*)

* possible to occasionally miss the time constraint on an event, as long as not too many are missed

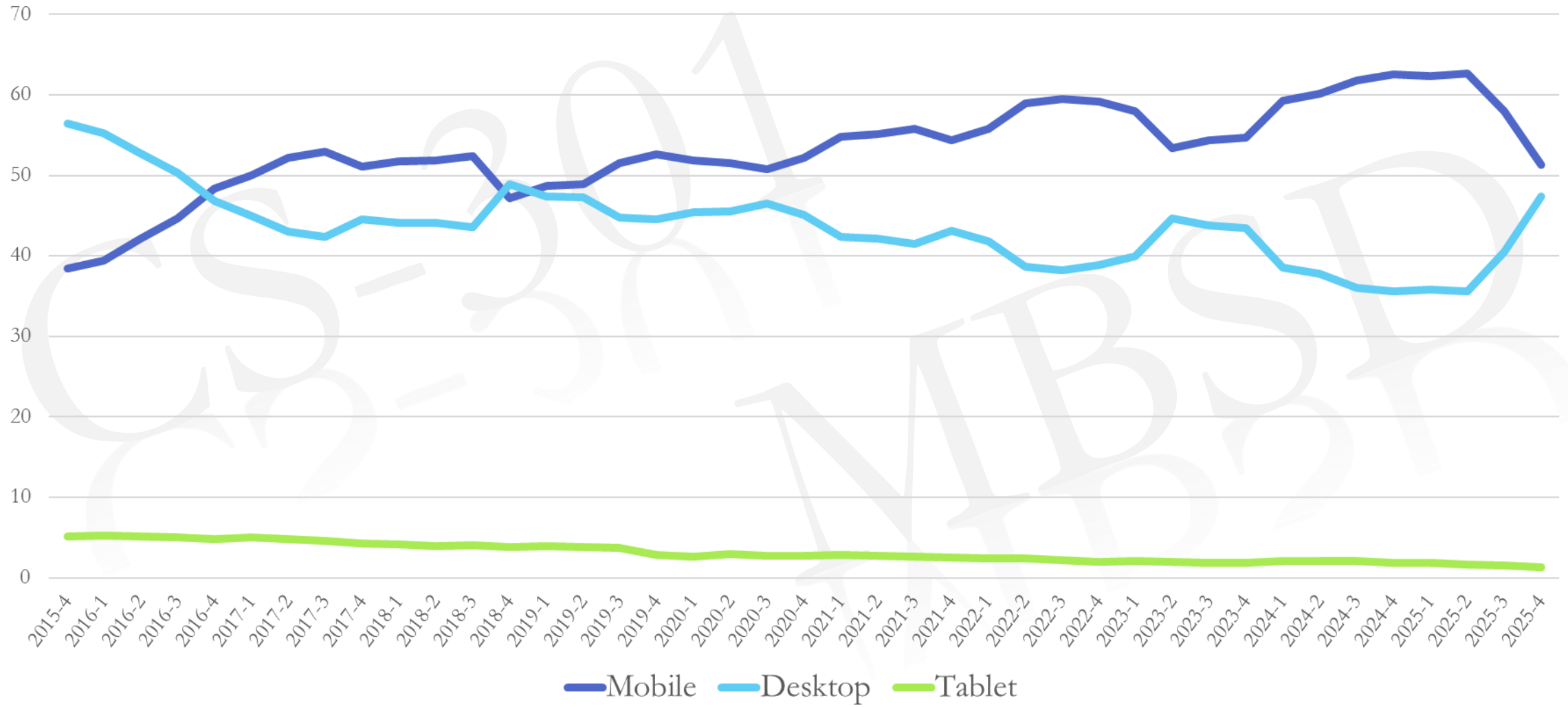
Classes of Computers

- Desktop Computing
  - 2nd largest market in terms of revenues
    - e.g. low-end netbook, laptops, PCs, heavily configured workstations
  - Well characterized in terms of applications and benchmarking
  - For the last 10 years, half of the desktop computers made each year are battery operated laptops

Emphasis on

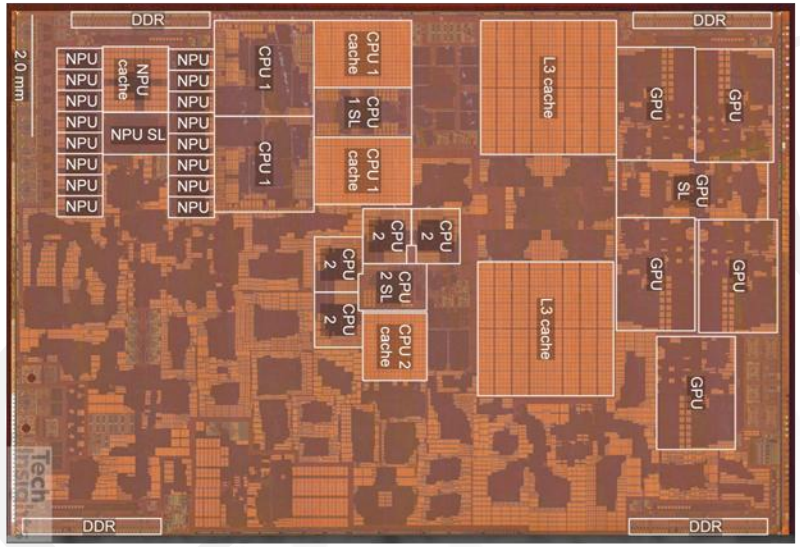
- Performance
- Cost
  - Price-performance combination
- Response time

Desktop vs Mobile vs Tablet



Mobile vs Desktop SoC

Apple A15 Bionic (PMD)



6 cores

- 4 energy efficient 2 GHz
- 2 high performance 3.2 GHz

32 MiB system cache (L3) Neural engine

15.8 TOP/s GPU, video, ISP

15 billion transistors TSMC 5 nm

3W normal 6W to 9W peak

Intel Alder Lake (Laptop/Desktop)

16 cores

- 8 energy efficient (E-core) 2.7 GHz
- 8 high performance (P-core) 5 GHz (turbo)

30 MiB L3 cache, AVX-512 SIMD instructions

Graphics & video accelerators, Memory, display,

PCI controllers 209 mm^2 die in Intel 7 nm process

20+ billion transistors, 240W peak

Classes of Computers

- Servers
  - Large scale and reliable file and computing services
  - Replacement of traditional mainframes

  Emphasis on
  - Availability (24/7 operational)
  - Scalability
    - As demand increases so should the computing capacity, the memory, the storage and I/O bandwidth of the server increase.
  - Throughput
    - Responsiveness to an individual request is important but efficiency is determined by how many requests can be handled in a unit time.

Classes of Computers

- Clusters / Warehouse Scale Computers
  - Used for "Software as a Service (SaaS)"
    - e.g. Search, Social Networking, Online shopping, Multiplayer games, file sharing etc.
  - Clusters are collection of desktop computers or servers connected by local area networks to act as a single larger computer
  - When tens of thousands of computers act as one the clusters are called warehouse- scale computers

Emphasis on

- Availability
- Price
- Performance
- Power
- Internet Bandwidth

Sub-class: Supercomputers, emphasis: floating-point performance and fast internal networks (usually in a close proximity)

Classes of Computers

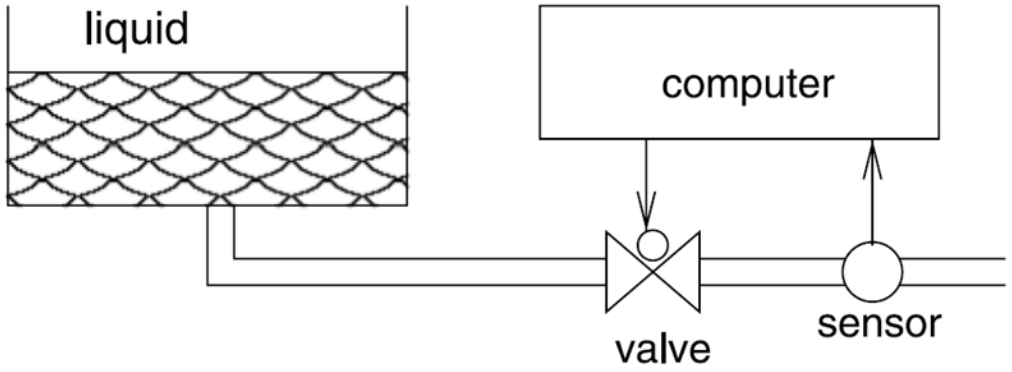
- Embedded Computers
  - Embedded systems are information processing systems that are embedded into a larger product.
    - Products like cars, trains, planes, medical equipment, buildings, telecommunications
  - PMDs are examples of embedded computers but can run externally developed software and so share some characteristics of desktop computers
  - Wide variety of processors (general purpose, special purpose, single purpose) and wide variety of applications

Emphasis on

- Power and Energy
- Cost
- Size and Weight
- Performance
- Real-time constraints
  - (both hard and soft)

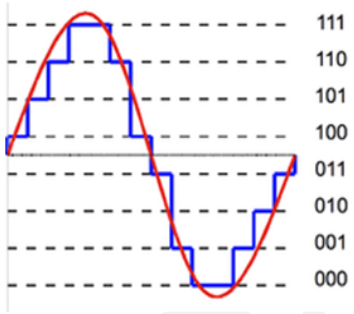
Embedded System Hardware

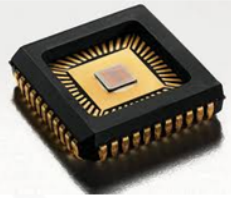
- Frequently, embedded systems are connected to the physical environment through **sensors** collecting information about that environment and **actuators** controlling that environment.
  - actuators are devices converting numerical values into physical effects.

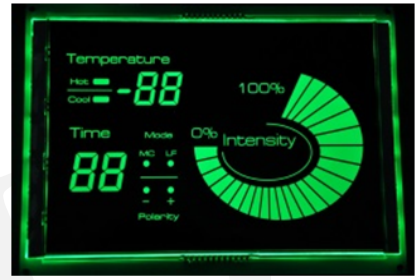


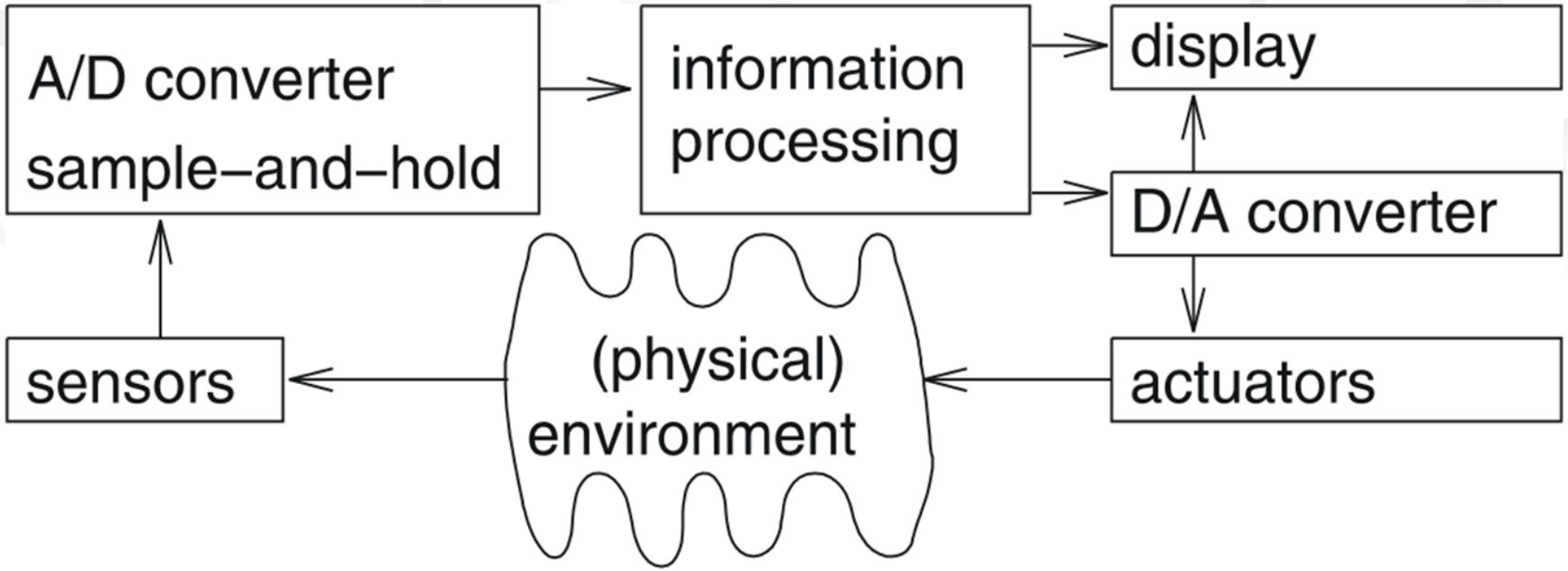
Embedded System Hardware

- Embedded system hardware is frequently used in a loop (***"hardware in a loop"***):









Course Outline

- System Design Metrics
  - NRE Cost, Time to Market, Performance
- Processor's Internal Architecture
  - Instruction cycle, Registers, Addressing modes, U74 architecture …
- System Bus
  - Bus timing, wait states, …
- Memory System
  - Memory mapping, memory management, …
- Peripherals
  - I/O interfacing, interrupt handling, Timer, PWM, DMA, ADC …
- Communication Protocols
  - UART, USB, I2C, SPI, Bluetooth …
- Processor Advanced Features
  - low power design, OS support features, multicore processor memory management

Course Learning Outcomes

Course Learning Outcomes (CLO)		Taxonomy Level	Program Learning Outcome (PLO)
CLO-1	Explore internal microprocessor architecture and operations	Cognitive C3 (Application)	PLO-1 Engineering Knowledge
CLO-2	Illustrate interfacing techniques of a microprocessor with memory and I/O devices	Cognitive C4 (Analysis)	PLO-2 Problem Analysis
CLO-3	Practice interfacing microprocessor with other system components	Psychomotor P3 (Guided Response)	PLO-3 Design/ Development of Solutions

Sessional Criteria

- Assignments/Quizzes (10)
- Midterm (20)
- Complex Engineering Problem/Project (10)

Recommended Books/Reading Material

- SiFive U74-MC Core Complex Manual
- Embedded System Design - Embedded Systems, Foundations of Cyber-Physical Systems and the Internet of Things, by Peter Marwedel, 3rd edition, Springer, 2018

CS-301 Microprocessor Based System Design

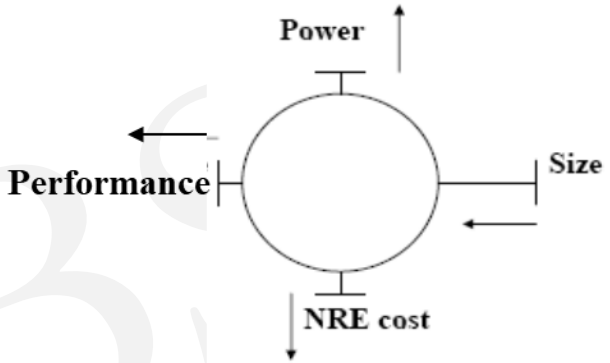
Course Teacher: Dr.-Ing. Shehzad Hasan

Lecture – 2

Design Metrics

- Design Metric
  - A measurable feature of the system's implementation e.g. power, performance, time, safety etc.
  - Metrics are indicators that show how well your system meets its goals and expectations.

- Improving one may worsen others
  - e.g. reducing power may degrade performance



- Before evaluating the embedded system's performance one needs to define the metrics to measure and why.

- Expertise with both **software and hardware** is needed to optimize design metrics
  - Not just a hardware or software expert, as is common

Design Metrics

- Accuracy
  - How accurate is the implementation of the desired functionality remaining within the required time constraints.
    - For a normal hardware based system it is expected to be 100% but if ML models are included then we need to know how closely the system's predictions or classifications match the true outcomes
    - ML models often provide probabilistic outputs, accuracy might be complemented by other metrics like precision, recall, or F1 score to provide a better view of system performance
- NRE cost
  - Non-Recurring Engineering cost
  - The one-time monetary cost of designing the system
- Unit cost
  - the monetary cost of manufacturing each copy of the system, excluding NRE cost

NRE & Unit Cost

- The total cost of your design is the sum of the non-recurring engineering cost and the number of units produced times the unit cost.

$$\text{Total cost} = \text{NRE cost} + (\text{unit cost})(\#\ \text{units})$$

- What should then be the per-product cost?

$$\text{per-product cost} = \text{total cost} \ / \ \#\ \text{units}$$

$$= (\text{NRE cost} / \#\ \text{of units}) + \text{unit cost}$$

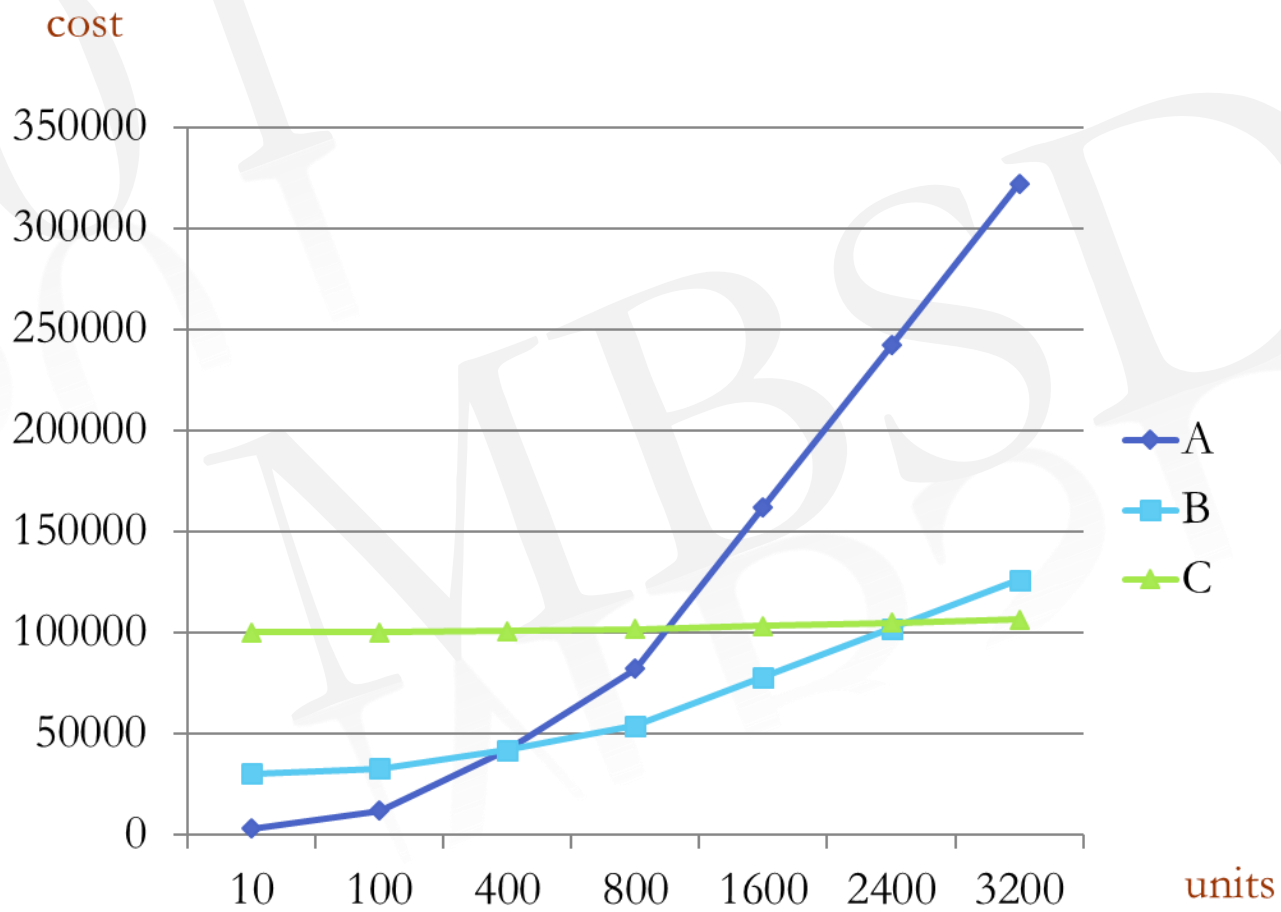
- for any technology, the per-product cost approaches unit cost for very large volumes

NRE & Unit Cost

Suppose three technologies are available for implementing a product

Technology	NRE Cost	Unit Cost
A	2,000	100
B	30,000	30
C	100,000	2

Total cost = NRE cost + (unit cost)(# units)



Design Metrics

- Performance
  - How long does the system take to execute the desired task
  - Widely-used measure of system, & widely-abused
    - Clock frequency (GHz), instructions per second (MIPS) – not a good measure
      - Digital camera example – a user cares about how fast it processes images, not clock speed or instructions per second
  - Latency (response time)
    - Time between the start of task's execution and the end
      - e.g., Cameras A and B process images in 0.25 seconds
  - Throughput
    - Number of tasks that can be processed per unit time,
      - e.g. Camera A processes 4 images per second
      - Throughput can be more than just inverse of latency. Due to either parallelism or concurrency, e.g. Camera B may process 8 images per second (by capturing a new image while previous image is being stored).
  - Speedup of B over A = S = B's performance / A's performance
    - Throughput speedup = 8/4 = 2

Design Metrics

- Power Consumption
  - the amount of power consumed by the system which may determine the lifetime of a battery or the cooling requirements of the IC since more power means more heat
- Size
  - the physical space required by the system often measured in bytes for the software and gates or transistors for hardware
- Flexibility
  - the ability to change the functionality of the system without incurring heavy NRE cost

Design Metrics

– Reliability

- Probability that the system operates correctly without failure for a specified period of time under stated conditions.

$$Reliability = R(t) = e^{-t/MTTF}$$

  - MTTF: Average time until the system fails
  - E.g. if 100 units collectively completed 350,000 hours before 20 failed, the MTTF equals 350000/20 = 17,500 hours per unit (reciprocal of failure rate(λ): failures/hour)
  - Reliability of the system to operate without failure for a month = R(720) = $e^{-720/17500}$ = 0.96

30 x 24 = 720

- We can increase the reliability by making the design fault tolerant
- Fault tolerance is the ability of an embedded system to continue correct operation despite the presence of faults. i.e. MTTF increases
  - The key thing here is that the fault is still going to exist but the system is not going to fail.
  - Typically we use some kind of redundancy (hardware, time, information, design diversity) to avoid system failure

CS-301 Microprocessor Based System Design

Course Teacher: Dr.-Ing. Shehzad Hasan

Lecture – 3

Design Metrics

- Availability
  - the fraction of time an embedded system is operational and providing service, considering both failures and repair/recovery time.

$$Availability = \frac{MTTF}{MTTF + MTTR}$$

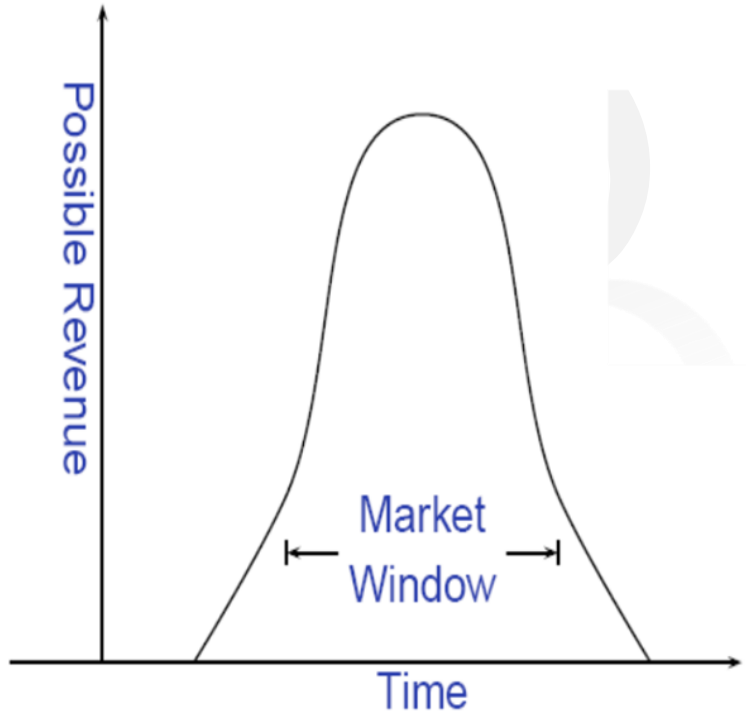
  - MTTR: Mean time to repair (or recover)
  - E.g. if for a sensor node the MTTF = 90 days (2160 hours) but can restart and recover in 1 min i.e. MTTR = 1 min (0.0167 hours) then Availability = 0.999992

Design Metrics

- Time-to-prototype
  - the time needed to build a working version of the system which may be bigger or more expensive than the final implementation.
- Time-to-market
  - commonly abbreviated as T2M, is the time required to develop a system to the point that it can be released and sold to customers (= design time + manufacturing time + testing time)

The Time-to-Market Challenge

- Market window
  - Period during which the product is in demand and would have highest sales
- Revenue curve
  - the revenue generated by a product or service over a specific period (market window), typically showing the growth and decline of revenue as the product goes through its life cycle
- Delays can be costly
  - In some cases, each day that a product is delayed from introduction to the market, can translate to millions of dollar loss!



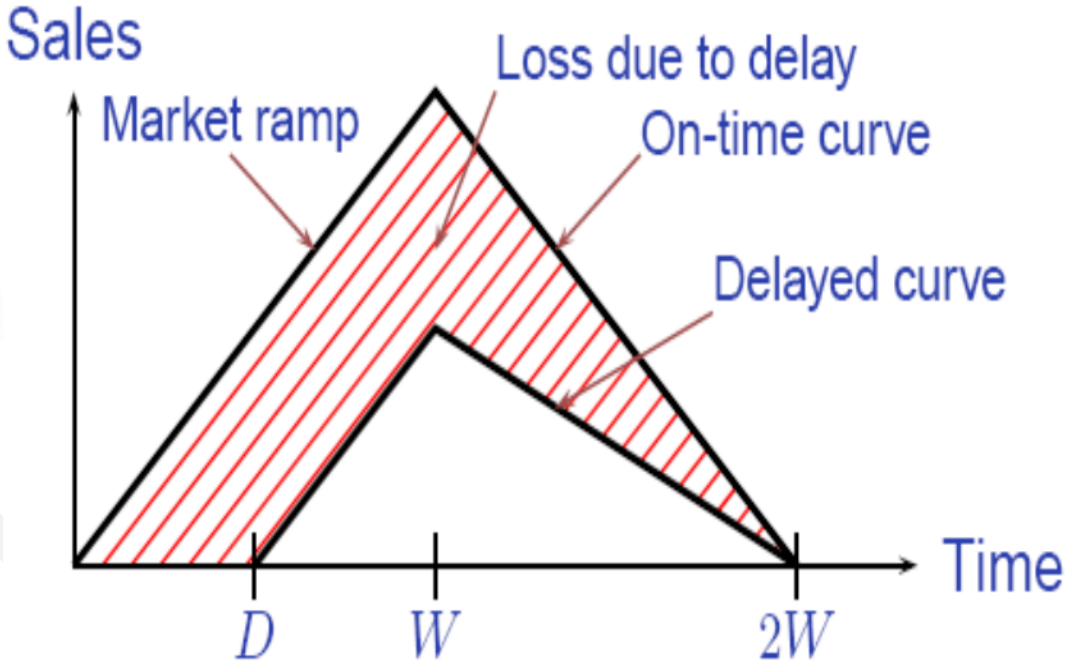
Revenue Curve

- For **embedded systems**, the most common revenue curve shape tends to be a bell-shaped pattern.
  - This is due to the nature of embedded systems, which often have longer adoption cycles, especially in industries like automotive, healthcare, or industrial automation where reliability and integration time are crucial. Here's how the revenue phases typically progress:

1. **Introduction Phase**: Slow initial growth as the technology is introduced, and early adopters begin to integrate it.
2. **Growth Phase**: Rapid acceleration as more companies adopt the technology, and it becomes more standardized.
3. **Maturity Phase**: Revenue stabilizes as the market reaches saturation and most potential customers have adopted the technology.
4. **Decline Phase**: A gradual decline might occur as newer technologies emerge and replace the existing systems.

Losses Due To Delayed Market Entry

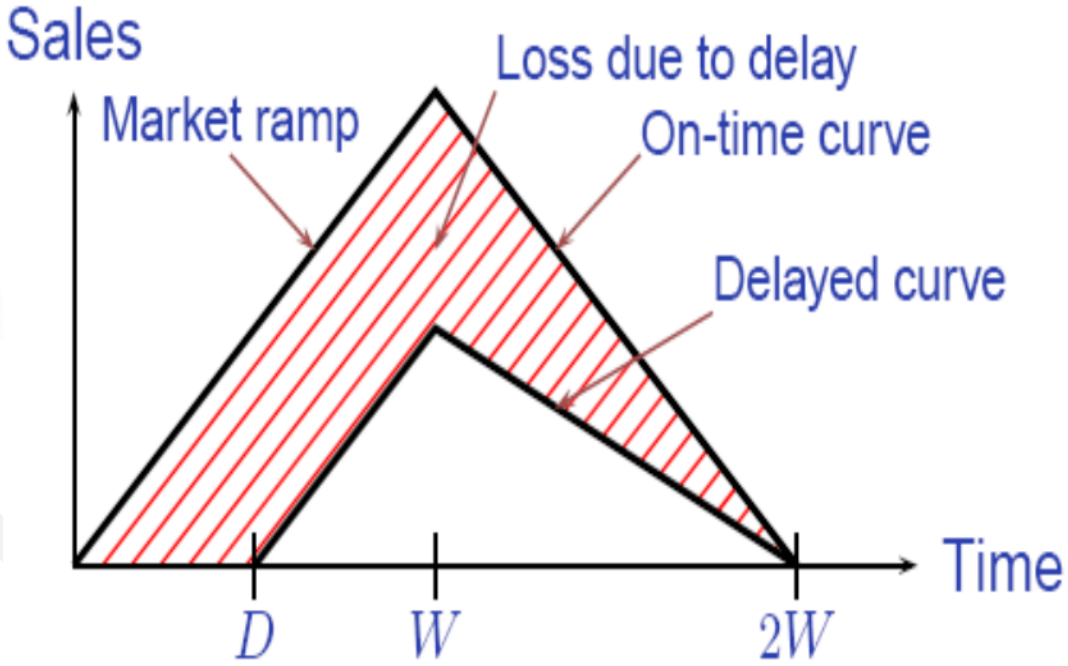
- Simplified revenue model
  - Product life = 2W,
  - peak at W
  - Time of market entry defines a triangle, representing market penetration
  - Triangle area equals revenue
- Loss
  - The difference between the on-time and delayed triangle areas



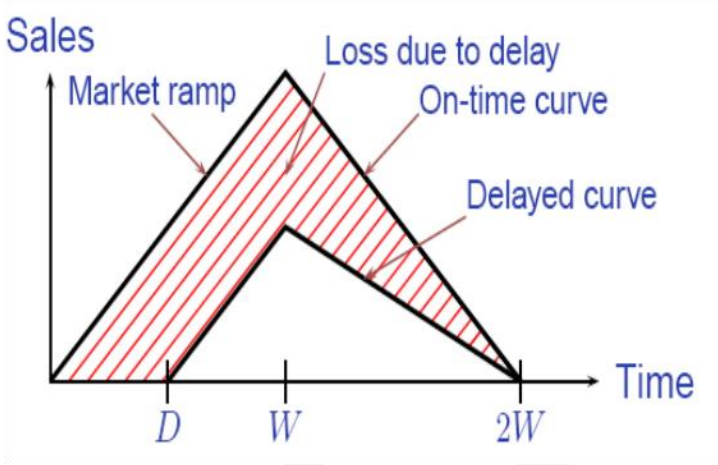
Lets consider the market ramp angle to be 45°

Losses Due To Delayed Market Entry

- Revenue?
  - Triangle's area
    - On-time Revenue?
      - **1/2 * 2W * W = W^2**
    - Delayed Revenue?
      - **1/2 * (2W-D)*(W-D)**
  - **Percentage revenue loss?**
    - $(D(3W-D)/2W^2)*100\%$
- **Examples**
  - **W=26,D=4**
    - **22%**
  - **W=26, D=10**
    - **50%**
- **Delays are costly!**



Do it Yourself



1. Using the simplified revenue curve, calculate the percentage revenue loss for any market ramp angle 'θ' other than 45 degrees.

2. The design of a particular disk drive has an NRE cost of $200,000 and a unit cost of $20. How much will we have to add to the cost of each product to cover our NRE cost, assuming we sell: (a) 100 units, and (b) 100,000 units?

Design Metrics

- Maintainability
  - The ability to maintain the system after its initial release, especially by designers who didn't originally design the system
- Safety
  - The likelihood that the system will cause no harm to the user or environment
- Correctness
  - Measure of confidence that we have implemented the system's functionality correctly.
  - We can check the functionality throughout the process of designing the system and can insert test circuitry to check that manufacturing was correct.

Design Metrics

Additional Design Metrics may include

- Security
  - How secure is the system?
  - Cybersecurity is the main concern these days specially with remotely accessed IoT devices
- Usability
  - How easy it is to use the system
  - This metric is specially useful if the product is intended for senior citizens or differently-abled people.
- Functionality in Harsh Environment
  - Heat, vibration, shock
  - Power fluctuations, RF interference, lightning
  - Water, corrosion, physical abuse